Lab 3: Mininet, OpenFlow, OpenVSwitch, Controllers (POX)

M2 CSN – SDN

Exercice 1 : First deployment

You are going to use an SDN controller to program specific behaviors.

For this, we will use the POX controller, developed in Python and oriented towards research and the rapid development of prototypes.

POX offers several "components" that can be used directly and it is relatively easy to design a custom one.

A POX component corresponds to a "network application" deployed on the controller.

See POX documentation at https://openflow.stanford.edu/display/ONL/POX+Wiki.html.

First, we will use an existing POX component corresponding to our needs.

— In a separate SSH window, run POX with the following command:

$ sudo python pox/pox.py openflow.of_01 --port=6633 forwarding.l2_learning

You should normally see the line appear at the end of the output:

INFO:core:POX 0.2.0 (carp) is up.

You must then keep this process running in the foreground throughout the experiment.

It is possible to load several components simultaneously in the same command line, but some are incompatible with each other.

In another window, launch Mininet in its default topology with the POX controller:

$ sudo min --controller=remote,port=6633

How are you making sure that the network starts correctly, that the output of POX indicates connecting a new switch, and that switch seems to behave in the expected way?.

Exercice 2: Gateway programmability using POX SDN Controller

You will now start both Mininet and POX from a single Python script. In this configuration, we will use two different SDN controllers in the same network (native mininet controller and POX controller), in order to have switches exhibiting different behaviors.

For this purpose, you will find in the annex the python script named **passerelle.py** to put in the /home/mininet directory. It is made up of two different parts:

Read these two part and explain what are the purpose of the first part and the second part ?

Run the script with the following command and respond to the question.

$ sudo python passerelle.py.

- Is the ping between machines working?
- Why it is not working?
- What should the POX controller do to change the behavior of switch c1 to solve this problem?

You could rely on the L3_learning component documentation:

https://openflow. stanford.edu/display/ONL/POX+Wiki.html#POXWiki-forwarding.l3_learning;

Modify the passerelle.py script to make POX, which drives c1, allow it act as a gateway between IPv4 networks;

Restart the execution;

- Why it still does not work?
- What changes to do in the h1 and h2 configurations to makes it work ?
- What is the configuration commands to execute directly from Mininet command line:

Make sure the ping is now working between h1 and h2.

Exercice 3: Multi-path routing POX component

You are now going to tackle a classic problem in datacenters networks, multipath routing. POX does not offer a component directly usable to solve this problem and the objective here is to develop this custom component.

Two Python scripts are used for this manipulation:

— You need to use the script diamond.py provided in the annex (launched with **$ sudo python diamond.py**), reuse the previously introduced CustomPOX and CustomSwitch classes and defining a particular network topology (in which you have fixed MAC addresses, IP addresses and port numbers on the switches).

Two controllers are started here, c which is an Open vSwitch controller (comparable to the OpenFlow reference controller) and cm, which is a POX controller loading the misc.tp custom component. In the

considered topology, two switches, one refers to c and two to cm. The diamond.py script should not be modified;

— You need to put the tp.py script (in annex) in the directory /home/mininet/pox/pox/misc/tp.py defines the misc.tp component. In his initial state, it configures the connected switches to function as hubs, i.e. to repeat frames on all ports except the input port.

The script to modify is the tp.py script that you will need to modify, and in particular its act_less_stupidly method.

How the tp.py script works is described in the source code comments. Read the comments and understand the general logic of each of the following function:

— __init__ (self, connection)

— launch()

— handle_PacketIn (self, event)

— act_like_hub (self, packet, packet_in)

— **act_less_stupidly (self, packet, packet_in)**

— resend_packet (self, packet_in, out_port)

To complete this exercise, you will need technical information on a few classes defined by POX. Available at https://openflow.stanford.edu/display/ONL/POX+Wiki.html (or at https://noxrepo.github.io/pox-doc/html/),

— ofp_packet_out

— ofp_flow_mod

— ofp_match

— ofp_packet_in

Draw a small diagram of the topology described in the script diamond.py;

Make sure you understand which switches are driven by which SDN controller;

Launch Mininet on the original file with **sudo python diamond.py**;

In a separate (console) window, start a packet capture on the interface s1-eth1 from switch s1, with:

$ sudo tcpdump -XX -n -i s1-eth1

In the command line of Mininet, launch a ping with a single exchange between h1 and h2, with

h1 ping -c 1 h2

Observe the output of the tcpdump command;

- What happened?
- Does the behavior of the network seem appropriate?
- Given the network topology and the specified switch behavior in the tp.py script, what is the problem?

In an office-type network (where performance requirements are not necessarily critical), this type of problem is frequently solved by using the Spanning Tree Protocol (STP), which will block certain links between switches to transform a graph into a tree and "open" all loops.

In a datacenter type network, we cannot afford it: if we have redundant links, with several possible paths between two points, it is to increase the bandwidth of the flows horizontal. We therefore rely on other mechanisms, by introducing multi-path routing. This is typically done using IEEE 802.1aq (Shortest Path Bridging) protocols or IETF TRILL (Transparent Interconnect of Lots of Links).

Without reproducing exactly the mechanisms of these protocols, you will try to modify the behavior of switches s3 and s4 to resolve this problem, while still benefiting from the additional bandwidth provided by the redundant links.

Designing the functional solution:

By modifying the **act_less_stupidly** method of the **tp.py** script, design a new mode of operation for switches s3 and s4.

The problem can be approached in an iterative way, by first proposing a solution solving the identified problem but without benefiting from multiple paths.

The final solution should have the following characteristics:

- Each of the hosts must be able to communicate with each of the others;
- Packets must not be duplicated in an uncontrolled manner or circulated indefinitely in the network;
- Overall, all the links between switches must be exploited (not necessarily in a balanced way) when all the hosts communicate between themselves.

There are many ways, more or less subtle and more or less complex, to reach the final solution. Each student may propose a different solution from the basic one to the advanced one.

Some hints: a basic solution will be to statically assign links to some hosts, to certain flows, to some protocols. A more advanced solution will be to design a behavior that is capable to evolve over time with variations in flows. A more advanced solution will be to design a behavior of switches that is capable to change with network topology.